

Energy Markets MCP Server — Toolset Design

Context

The portfolio dashboard already analyses energy-sector equities (AR, AM, NEE, BWXT, CCJ, OKLO, CMI, EOG, GEV, VST, CEG, TLN, etc.) and has a saved macro framework in memory connecting PJM/SERC megawatt additions to the utility/IPP/nuclear thesis. But the dashboard has no first-class access to **commodity-side data**: natural gas spot/curve, storage, LNG exports, rig counts, power prices, capacity auction prints, or weather demand drivers. Today, any "natural gas thesis" analysis has to rely on the equity tape as a noisy proxy.

This plan designs a new `energy-markets-server` that exposes the commodity- and grid-side signals as MCP tools, mirrors the existing server patterns (FastMCP + SQLite cache + TTL freshness checks), and integrates with the existing trade-recommendation engine so energy-tagged tickers get scored against their own commodity backdrop rather than only generic technicals.

The goal of this document is to specify the **API surface** of the new server — what tools it exposes, where the data comes from, and how it fits into the existing architecture. Implementation is a follow-up.

Proposed server

File: `fastMCPTest/energy_markets_server.py` **Name:** `energy-markets-server` **Pattern:** Mirrors `market_analysis_server.py` and `company_fundamentals_server.py` (FastMCP single-file server, SQLite cache neighbor, TTL freshness checks).

Companion helper: `fastMCPTest/energy_markets_cache.py` — SQLite persistence (`energy_markets.db`) following the `ohlcv_cache.py` + `fundamentals_cache.py` pattern (WAL mode, double-checked locking, append-only history tables).

Tool inventory (phased)

Phase 1 — MVP (6 tools)

Goal: cover the core commodity + storage + activity signals that drive the natural gas thesis.

Tool	Purpose	Source
<code>get_natural_gas_price</code>	Henry Hub spot + nearest-month NYMEX futures	EIA API (<code>NG.RNGWHHD.D</code>) + <code>yfinance</code> / <code>NG=F</code>
<code>get_natural_gas_storage</code>	Weekly working gas storage, delta vs 5-yr avg + 1-yr ago	EIA API (<code>NG.NW2_EPG0_SW0_R48_BCF.W</code>)
<code>get_crude_oil_price</code>	WTI + Brent spot + nearest-month futures	EIA API + <code>yfinance</code> <code>CL=F</code> , <code>BZ=F</code>
<code>get_crude_oil_inventory</code>	Weekly EIA petroleum status — commercial crude stocks	EIA API (<code>PET.WCESTUS1.W</code>)
<code>get_rig_count</code>	Baker Hughes weekly U.S. gas + oil rig count	Baker Hughes scrape / EIA mirror

Tool	Purpose	Source
<code>get_natural_gas_consumption</code>	Monthly total + by sector (residential / commercial / industrial / power)	EIA API (NG . N9140US2 . M series)

Phase 2 — Supply / demand depth + power markets (8 tools)

Tool	Purpose
<code>get_natural_gas_curve</code>	12-month NYMEX futures strip; identifies contango/backwardation
<code>get_lng_exports</code>	Monthly LNG export volumes by terminal + destination
<code>get_natural_gas_production</code>	Monthly U.S. dry gas production by region (Appalachia, Permian, Haynesville)
<code>get_power_price_dayahead</code>	Wholesale electricity day-ahead LMP for PJM West Hub, ERCOT Houston, CAISO SP15
<code>get_power_generation_mix</code>	Daily/weekly U.S. generation mix (gas / coal / nuclear / wind / solar / hydro)
<code>get_pjm_capacity_auction</code>	Most recent base residual auction results — \$/MW-day clearing prices
<code>get_heating_degree_days</code>	HDD/CDD current + 14-day forecast vs 30-year norms (major demand regions)
<code>get_crack_spread</code>	3-2-1 crack spread (refining margin proxy)

Phase 3 — Cross-asset signals & watchlist integration (6 tools)

These synthesize commodity data with the existing equity/options tape.

Tool	Purpose
<code>get_natgas_oil_ratio</code>	Henry Hub ÷ WTI (Btu economics) — drives substitution thesis
<code>get_energy_sector_signal</code>	Commodity vs XLE divergence — flags rotation opportunities
<code>get_natgas_utilities_signal</code>	Natgas curve vs XLU divergence — quality check for NEE/CEG/VST type theses
<code>get_pjm_interconnection_queue</code>	Queue size, recent additions, withdrawal rate by tech (gas/solar/storage/nuclear)
<code>analyze_energy_watchlist</code>	Score energy-tagged watchlist names against current commodity backdrop

Tool	Purpose
<code>get_natgas_exposure_screen</code>	Rank watchlist by historical sensitivity (beta) to natgas spot moves

Data sources & rationale

Primary: EIA API (api.eia.gov/v2). - Free, requires API key - Comprehensive: gas, oil, coal, power, renewables, LNG - Limits: 5,000 req/hour (well within usage) - Lag: weekly series (Thursday for storage, Friday for rigs) — must label "as of" dates

Secondary: yfinance (already integrated). - Futures proxies: `/NG=F`, `CL=F`, `BZ=F`, `RB=F`, `H0=F` - Commodity ETFs already in watchlist contexts: UNG, USO, BOIL, KOLD - Reuse existing `ohlcv_cache.py` for futures/ETF history

Tertiary: NOAA (weather/HDD). - Free API - Only needed for Phase 2; can defer or stub with EIA's degree-day data first

Power markets: PJM API (free for basic auction/queue data) + EIA's hourly grid data.

No paid feeds in scope. Pipeline flow data, real-time LNG cargo tracking, and intraday natgas — all paid; explicitly out of scope.

Caching strategy

Mirror the `fundamentals_cache.py` TTL-checked append-only pattern. SQLite WAL mode, double-checked locking on init, `_safe_int/_safe_float` helpers for NaN safety (lesson learned from the recent `options_contract_tools.py` bug fix).

Schema (`energy_markets.db`):

Table	Granularity	TTL (env var)	Notes
<code>commodity_prices</code>	daily	<code>ENERGY_PRICE_CACHE_TTL_HOURS</code> (default 4)	Spot + nearest futures
<code>commodity_curves</code>	daily snapshot, 12 months forward	4h	Full strip persistence for backtest
<code>storage_reports</code>	weekly	24h (releases Thu)	EIA gas + crude inventories
<code>production_data</code>	monthly	7d	Gas/oil production
<code>consumption_data</code>	monthly	7d	Demand by sector
<code>rig_counts</code>	weekly	24h (releases Fri)	Baker Hughes
<code>power_prices</code>	hourly day-ahead	1h	LMPs by hub
<code>power_generation_mix</code>	daily	24h	EIA grid data

Table	Granularity	TTL (env var)	Notes
capacity_auctions	annual	30d	PJM auction prints
lng_exports	monthly	7d	EIA cargo data
weather_demand	daily + 14d forecast	6h	HDD/CDD

TTL env vars read on every call (per the existing `FUNDAMENTALS_CACHE_TTL_HOURS` pattern) so they can be tuned without restart.

Cross-server integration

1. stock-price-server → consume energy signals in `get_trade_recommendation`.

Add a new signal to the 19-signal scorer: when ticker has tag `Energy` or `Utilities` or `Oil & Gas`: - Pull `get_natgas_oil_ratio` and current curve state - Add +1 bull point if natgas spot above 60-day MA *and* the ticker is natgas-levered (AR, AM, EOG with gas weight) - Add +1 bear point if natgas spot below 30-day low

This piggybacks on the existing pattern where `stock_price_server.py` already imports from `market_analysis_server.py` and `company_fundamentals_server.py`.

2. options-analysis-server → reweight options scoring for energy names.

In `analyze_options_watchlist`, when scoring energy tickers: - Pull `get_natural_gas_storage` deviation from 5-yr avg - If storage deficit > 15%, add +1 to `long_score` for gas-levered names - If storage surplus > 15%, add +1 to `put_score`

3. company-fundamentals-server → cross-reference for sector breakdown.

Optional: `get_sector_fundamental_breakdown(sector="Energy")` could overlay current commodity backdrop as context.

Configuration

Add to `.env`:

```
EIA_API_KEY=<key>
# Optional tuning (defaults in code)
ENERGY_PRICE_CACHE_TTL_HOURS=4
ENERGY_STORAGE_CACHE_TTL_HOURS=24
ENERGY_RIG_CACHE_TTL_HOURS=24
ENERGY_LNG_CACHE_TTL_DAYS=7
ENERGY_POWER_CACHE_TTL_HOURS=1
```

Register the server in `.mcp.json`:

```
"energy-markets-server": {
  "command": ".venv/bin/fastmcp",
  "args": ["run", "fastMCPTest/energy_markets_server.py"]
}
```

Add `"energy-markets-server"` to `enabledMcpjsonServers` in `.claude/settings.local.json`.

Tests

Mirror `test_options_contract_tools.py` pattern (unittest + tempdir fixtures + injected fetchers).

New test files (project root): - `test_energy_markets_cache.py` — TTL freshness, schema migrations, NaN safety - `test_energy_markets_tools.py` — each Phase 1 tool with a mocked EIA response fixture - `test_energy_cross_integration.py` — verifies stock-price-server's `trade_recommendation` picks up energy signals when the ticker has an Energy tag

Add Bash permission allowlist entries in `.claude/settings.local.json` for the new test commands.

Documentation

- Add a new row block in `docs/capabilities-matrix.md` for each tool, showing MCP / REST / WebUI / CLI coverage (initially MCP-only)
 - Add an architecture entry in `fastMCPTTest/README.md` table for the new server file + cache db
 - Write `docs/proposals/ENERGY_MARKETS_PROPOSAL.md` describing the data source choices, scoring rules added to existing engines, and backtest methodology (mirroring the existing `OPTIONS_ANALYTICS_PROPOSAL.md` precedent)
-

Critical files this plan would create or modify

New files: - `fastMCPTTest/energy_markets_server.py` — the server - `fastMCPTTest/energy_markets_cache.py` — SQLite persistence helper - `test_energy_markets_cache.py` - `test_energy_markets_tools.py` - `test_energy_cross_integration.py` - `docs/proposals/ENERGY_MARKETS_PROPOSAL.md`

Modified files (Phase 3 integration only): - `fastMCPTTest/stock_price_server.py` — extend `get_trade_recommendation` scorer (lines ~1100-1300 area, where the 19-signal sum happens) - `fastMCPTTest/options_analysis.py` — extend `analyze_options_watchlist` scoring (`long_score` / `put_score` boosts for energy tags) - `.mcp.json` — register server - `.claude/settings.local.json` — enable server + permission allowlist - `.env` — `EIA_API_KEY` and TTL knobs - `docs/capabilities-matrix.md` — new rows

Existing modules to reuse (do not reinvent): - `fastMCPTTest/ohlcv_cache.py: get_history()` — for any futures/ETF series (avoid yfinance dupes) - `fastMCPTTest/fundamentals_cache.py` — TTL pattern to mirror - `fastMCPTTest/options_contract_tools.py: _safe_int, _safe_float` — NaN safety helpers - `fastMCPTTest/stock_price_server.py: _SECTOR ETFS` — already maps Energy→XLE, Utilities→XLU

Phasing rationale

Phase 1 alone unlocks the core thesis work: "is natgas spot above support, is storage deficit/surplus, is rig count rolling over?" Six tools answer that. Most user analyses of AR/AM/NEE-style names would benefit immediately.

Phase 2 deepens the picture with forward curve, power markets, and weather. Necessary for the PJM/SERC tailwind framework already in memory to have real data behind it.

Phase 3 is where the leverage compounds — the cross-asset signals and watchlist integration make the new server *change how existing tools score* rather than just adding parallel queries.

Each phase is independently shippable. Recommend hard-stopping after Phase 1 to validate the pattern (cache freshness, EIA API reliability, integration test mechanics) before scaling out.

Verification

After implementation, end-to-end checks:

1. **Server health:** `claude mcp list` shows `energy-markets-server`: ✓ Connected
 2. **Smoke test:** Call `get_natural_gas_price` — returns dict with `spot_price`, `nearest_futures`, `as_of`, `signal`, `interpretation`
 3. **Cache:** call same tool twice within TTL — second call hits cache (verify via SQLite row count not changing)
 4. **TTL expiry:** set `ENERGY_PRICE_CACHE_TTL_HOURS=0`, recall — fetches fresh
 5. **Cross-server integration (Phase 3):** call `get_trade_recommendation` on AR; verify drivers list now includes a natgas line item when storage is in deficit
 6. **Unit tests:** `.venv/bin/python -m unittest test_energy_markets_cache test_energy_markets_tools test_energy_cross_integration -v` — all pass
 7. **No regression:** `.venv/bin/python -m unittest discover` — existing tests still pass
-

Open questions for the user

- **EIA API key:** willing to obtain one (free at [eia.gov](https://eia.doe.gov/))?
- **Phasing:** ship all three phases or stop at MVP and reassess?
- **Power markets scope:** PJM + ERCOT + CAISO for v1, or PJM-only and expand later? PJM alone covers the NoVA data center thesis cleanly.
- **Backtesting:** should commodity data be persisted append-only (like fundamentals) for future backtests of energy-thesis trades, or is point-in-time fetch sufficient?
- **Watchlist auto-tagging:** should the new server propose tag updates (e.g. flag AR as "Gas-Levered" with a beta-to-natgas value) or stay purely query-side?